Image Similarity with Swin Transformer

Veena Vyshnavi Garre[1], Itzel Xu[2], and Amruta Adusumilli[3]

[1]Department of Computer Engineering, San Jose State University

December 18, 2024

Abstract

Image similarity detection is crucial for image search and recommendation systems. Traditional methods often fail in capturing complex features and understanding image context and semantics. Traditional methods have shown limitations in capturing multivariate features and understanding image context and semantics compared with neural network methods. This project leverages the Swin Transformer to enhance content-based image retrieval (CBIR) systems, addressing these limitations. We utilize the CIFAR-100 dataset and employ triplet loss to retrain a pre-trained Swin Transformer model by converting images into anchor, positive, and negative triplets, with cosine similarity used to evaluate image similarity. Our findings indicate that the weights of the patch merging and attention layers do not change significantly. Additionally, the t-SNE visualizations of the output embeddings do not show distinct clusters. Results suggest that image size of the retrained dataset should be equal to or larger than the pretrained dataset is necessary to better capture features.
Keywords - Image similarity, Swin Transformer, CIFAR-100, Triplet loss

1 Introduction

1.1 Context/Background

Image matching has been studied for a long time in image processing and computer vision. It plays an important role in image searching and recommendation systems (Paul, n.d.). In traditional image processing, methods such as cosine metric, histograms, SSIM (Jim Nilsson, 2020), and SIFT (Lowe, 2004) are used which can solve most of the problems, but they are limited in capturing multivariate features and lack a deeper understanding of image context and semantics. On the other hand, machine learning methods, such as Siamese Network (Koch et al., 2015), Autoencoder (Adrian, 2020, and Vison Transformer (Dosovitskiy et al., 2021) have shown great performance in image similarity detection with their ability to learn complex features. Among them, the transformer with its multi-head self-attention and encoding block to produce embedding and decoder to explain those features makes it outstanding in dealing with visual problems (Paul, 2023). Yet Transformer cannot capture multi-scale features, and its calculation among global attention makes it computationally costly. Meanwhile, Swin transformer addresses these problems with hierarchical architecture and reduces computational load as well as maintaining high performance in capturing image context. It has demonstrated high performance in various computer vision tasks, making it a promising approach for image similarity detection.

1.2 Goals/Objectives

The goal of this project is to develop a Content Based Image Retrieval system using Swin Transformer model. CBIR is computer vision based image search application where it searches for similar images based on the input image. The brief steps of the application include extracting features of an input image. Later the extracted features are used to search for similar images with similar features from a large dataset of images.

It is widely used in e-Commerce platforms where people search for similar products using image search. It is also used in other domains like art, real estate, medical imaging, and travel.

This goal focuses on using the Swin Transformer to improve both the accuracy of image matching and the overall performance of the retrieval system, making it practical for large-scale deployment.

2 Related Work

2.1 Traditional Method

Traditional methods for image similarity detection compute directly between images using pixels, feature vectors, and their patterns or structures. These methods are computationally efficient but often limited in scope or precision. **Cosine Similarity**, which compares the angles between feature vectors, is especially useful for our model, as it effectively compares texture, color, shapes and patterns, making it suitable for matching preferences (Sitikhu et al., 2019). The reliance on the quality of the feature of this method ensures detailed and nuanced similarity detection, making it an ideal choice for our application (Antoniadis, 2023).

2.2 Neural Network Methods

Neural network methods capture comprehensive context and, while more computationally intensive, solve a wide range of problems with high versatility and accuracy. **Triplet Loss**, used in Siamese Networks, enhances our model's performance by learning similarity through contrastive learning with anchor, positive, and negative samples. This method excels in fine-grained similarity detection and is effective for image retrieval and classification tasks, providing robust and accurate feature representation (Hazem Essam, 2021).

3 Problem Statement

Traditional image search applications often rely on Convolutional Neural Networks (CNNs) as they are the backbone of many image retrieval systems. However, CNNs use fixed-size inputs, leading to the loss of important features and necessitating image resizing. This limitation becomes more pronounced as the dimensions of the input image increase, resulting in higher computational resource and memory requirements.

To address these challenges, we propose using a pre-trained Swin Transformer model combined with Triplet Loss as the loss function. Swin Transformers, with their hierarchical design and shifted window approach (Liu et al., 2021), are well suited for high-resolution images and are adaptable to various input image sizes while maintaining performance. They effectively capture both local and global features, making them a versatile choice for image retrieval tasks. Triplet loss further enhances the model performance by learning similarity through contrastive learning with anchor, positive, and negative samples, leading to robust and accurate feature representation.

This approach aims to leverage the strengths of the Swin Transformer and Triplet Loss to improve the accuracy and efficiency of content-based image retrieval systems.

4 Solution

4.1 Dataset

Effective image similarity models require careful dataset selection. In this project, we use a large labeled image dataset such as CIFAR-100 which provides a diverse range of images with various objects, scenes, and features. The dataset are essential for training and benchmarking image recognition and similarity tasks.

4.2 Swin Transformer Architecture

Our solution leverages the Swin Transformer (SwinT) model 1, which is built on a hierarchical design and uses a shifted windows approach (Liu et al., 2021). This architecture is highly adaptable to different input image sizes and performs well with high-resolution images. SwinT captures both local and global features efficiently, making it ideal for image retrieval tasks.

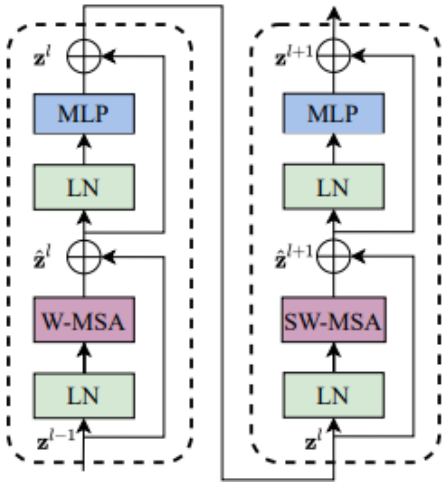

Figure 1: Swin Transformer Block (STB)

4.3 Training Procedure

To tailor the SwinT model for our specific image retrieval task, we employ the following steps:

Transfer Learning: We initialize the SwinT model with pre-trained weights from ImageNet, enabling the model to leverage learned features from large datasets, thereby reducing training time and enhancing performance.

Loss Function: We train the model using Triplet Loss (1), which ensures that similar images are close in the embedding space while dissimilar images are far apart. This involves using anchor, positive, and negative images to effectively capture image similarity.

Optimization: We use the AdamW optimizer with a cosine annealing learning rate schedule to minimize the loss. Regularization techniques like data augmentation and normalization are employed to improve model robustness and prevent overfitting.

Batch Size and Epochs: The model is trained with a batch size of 32 over 5 epochs, balancing memory constraints and convergence speed, ensuring adequate learning time.

$$L(A, P, N) = \max\left(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \text{margin}, 0\right) \tag{1}$$

A: Anchor image

P: Positive image (similar to the anchor)

N: Negative image (dissimilar to the anchor)

$f(x)$**:** Feature vector of image x

margin: A margin that enforces a minimum difference between the positive and negative pairs

4.4 Evaluation Metrics

The performance of our image retrieval system is evaluated using the following metrics:

Cosine Similarity: Measures the cosine of the angle between two vectors in the embedding space. This metric is used to assess the similarity between image embeddings, where higher values indicate greater similarity.

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\|\|B\|} \tag{2}$$

Euclidean Distance: Measures the distance between two vectors in the embedding space. Lower values indicate higher similarity.

$$\text{Euclidean Distance}(A, B) = \sqrt{\sum_{i=1}^{n}(A_i - B_i)^2} \tag{3}$$

Normalized Discounted Cumulative Gain (NDCG): Evaluates the quality of the ranked results by considering the positions of the relevant images.

$$\text{NDCG} = \frac{DCG}{IDCG}, \quad DCG = \sum_{i=1}^{p} \frac{2^{rel_i} - 1}{\log_2(i+1)} \tag{4}$$

Receiver Operating Characteristic (ROC) Curve and Area Under the Curve (AUC): Evaluates the performance of the retrieval system as a binary classifier.

$$\text{AUC} = \int_0^1 \text{TPR}(FPR)\, d(\text{FPR}) \tag{5}$$

These metrics provide a comprehensive evaluation of the model's performance, focusing on similarity measures and ranking quality.

Visualizing Feature Embeddings: We use t-SNE to visualize feature clusters in a 2D space, helping to qualitatively assess the embedding quality.

4.5 Implementation Steps

Our method involves the following steps 2:

1. Retrain the SwinT model using Triplet Loss.
2. Use cosine similarity to sort similarity scores.
3. Retrieve the top-k indices.
4. Display the query and the top-k images as illustrated.

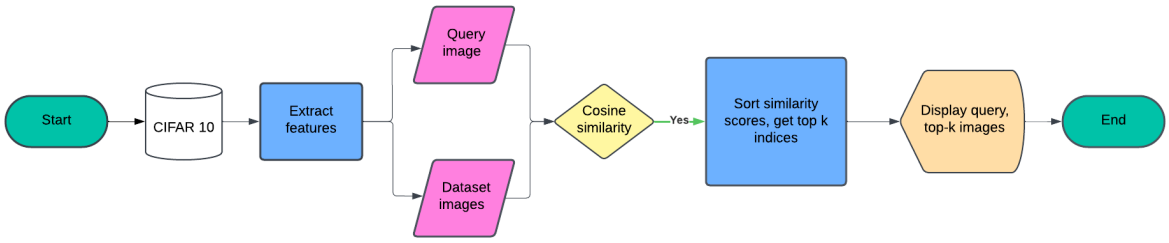

Figure 2: Flowchart of the Process

This approach combines the strengths of SwinT, Triplet Loss, and cosine similarity to enhance the accuracy and efficiency of our image retrieval system.

5 Results

5.1 Similarity Output

Image4 shows the similarity output for a given query image of a cow. The model retrieved images of a frog, dog, and cat, which are only similar in color but not in class. This indicates that the model's current feature extraction is more focused on color similarity rather than class similarity.

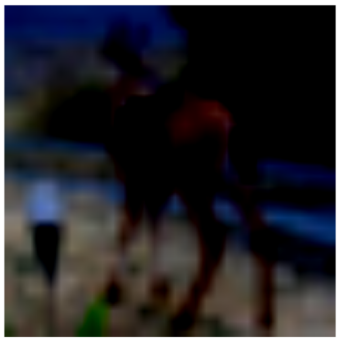

Figure 3: Input query

Figure 4: Top 3 similarity of query

5.2 t-SNE Feature Embedding

Image5 shows the t-SNE feature embedding for the output. The visualization does not display obvious clusters, indicating that the embeddings do not separate different classes well. This suggests that the model's ability to discriminate between different classes is inadequate.

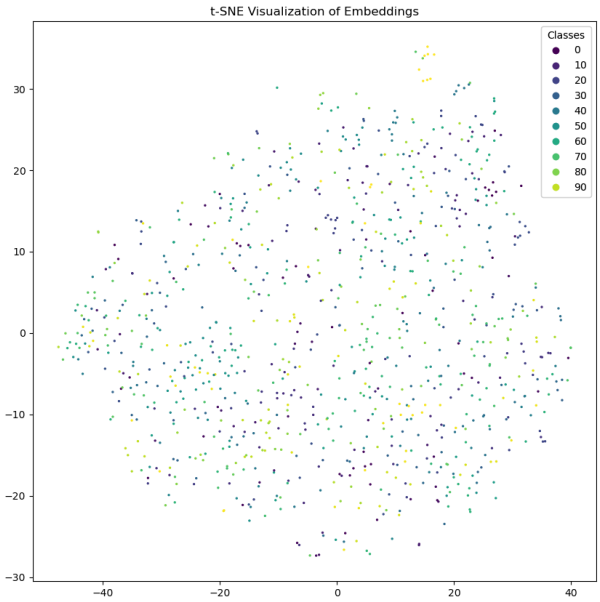

Figure 5: Flowchart of the Process

6 Findings

Training Time: The model was trained for 2 days over 5 epochs.

Loss: The loss decreased from 2.6597 to 0.7820, showing some improvement but not substantial.

Computation Time: Each image took approximately 1.41 seconds to process.

Weight Changes: The weights of the model barely changed after training.

Memory Usage: High memory usage was observed, resulting in RAM out-of-memory errors in Colab. On an AMD RTX 4050 GPU, the memory allocated was 608.36 MB, with 16540.00 MB reserved.

6.1 Solutions

For high memory usage -

Reduce Batch Size: Lower the batch size to fit within memory constraints.

Mixed Precision Training: Utilize mixed precision training to reduce memory usage.

Use More GPUs: Distribute the workload across multiple GPUs to handle larger datasets.
For bad performance -

Use High-Resolution Images: Higher resolution images may capture more detailed features, improving model performance.

Layer Freezing: Freeze certain layers during training to stabilize the learning process and focus on specific layers.

Increase Training Epochs: Train the model for more epochs to allow it to learn better. Adjust Learning Rate: Fine-tune the learning rate to find an optimal value that improves convergence.

7 Model Optimization

Use High-Resolution Images: Higher-resolution image training would make the model capture finer details and hence could improve feature extraction and semantic understanding.

More Training Epochs: It may further refine feature representations by extending the training to more epochs while overfitting is under control by appropriate regularization.

Learning Rate Tuning: Tweak the learning rate and/or use a learning rate schedule, for example, the warm restarts, to study the optimization of convergence.

Layer Freezing and Fine-Tuning: Freeze the lower layers first and fine-tune only the higher layers to capture task-specific features.

Data Augmentation: Employ more sophisticated data augmentation techniques to increase the variety of training data, including color adjustments, rotation, cropping, and flipping.

8 Computational Optimization

Mixed Precision Training: Train with half-precision (FP16) to reduce memory usage and accelerate computation.

Smaller Batch Sizes: Train with smaller batch sizes if the GPU memory is constrained, though this may slow convergence slightly.

Distributed Training: Scale training across multiple GPUs to handle larger datasets more efficiently.

9 Improvements in Evaluation

Cross-Domain Testing: The model should be evaluated on other datasets, such as sub-datasets of ImageNet, for further robustness and generalization.

Advanced Visualization: In addition to t-SNE, UMAP should be used for feature embedding visualizations to better understand the clustering of high-dimensional data.

Class-Aware Metrics: Include precision-recall metrics on specific image classes to study the quality of retrieval per category.

10 Acknowledgements

Thank Professor Gautam Krishna for guiding our team along the way. We would like to thank the contributors of the open-source projects and datasets that have significantly aided in the development of this work. Special thanks to the authors of the Swin Transformer and the CIFAR-100 dataset. The source code for our project can be found at the following GitHub repository: https://github.com/Veenasjsu/CMPE252-ImageSimilarity.

References

Adrian, R. (2020). *Autoencoders with keras, tensorflow, and deep learning.* Retrieved from https://pyimagesearch.com/2020/02/17/autoencoders-with-keras-tensorflow-and-deep-learning/ (Accessed: 2023-10-06)

Antoniadis, P. (2023). *Image comparison algorithms.* Retrieved from https://www.baeldung.com/cs/image-comparison-algorithm (2023-10-06)

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... Houlsby, N. (2021). *An image is worth 16x16 words: Transformers for image recognition at scale.* Retrieved from https://arxiv.org/abs/2010.11929

Hazem Essam, S. L. V. (2021). *Siamese network.* Retrieved from https://keras.io/examples/vision/siamese_network/ (Accessed: 2023-10-06)

Jim Nilsson, T. A.-M. (2020). *Understanding ssim.* Retrieved from https://arxiv.org/abs/2006.13846

Koch, G., Zemel, R., & Salakhutdinov, R. (2015). Siamese neural networks for one-shot image recognition. *Proceedings of the 32nd International Conference on Machine Learning*, 41-50. DOI: 10.5555/3305890.3305896

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., ... Guo, B. (2021). *Swin transformer: Hierarchical vision transformer using shifted windows.* Retrieved from https://arxiv.org/abs/2103.14030

Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, *60*(2), 91-110. DOI: 10.1023/B:VISI.0000029664.99615.94

Paul, S. (2023). *Image similarity.* Retrieved from https://huggingface.co/blog/image-similarity (Accessed: 2023-10-06)

Sitikhu, P., Pahi, K., Thapa, P., & Shakya, S. (2019). *A comparison of semantic similarity methods for maximum human interpretability.* Retrieved from https://arxiv.org/pdf/1910.09129